

Санкт-Петербургский государственный университет

Кафедра системного программирования

Группа 24.М41-мм

Подсистема обнаружения сигналов с цифровой модуляцией, реализованная на ПЛИС

Фаст Никита Михайлович

Отчёт по преддипломной практике
в форме «Производственное задание»

Научный руководитель:

Профессор кафедры системного программирования, д. т. н., Д. В. Кознов

Консультант:

Начальник отдела программирования АО «Концерн ГРАНИТ», А. С. Кривоногов

Санкт-Петербург
2026

Оглавление

Введение	3
1. Постановка задачи	6
2. Обзор	7
2.1. Особенности процесса разработки под ПЛИС	7
2.2. Используемые технологии	8
2.3. Существующие решения	10
2.4. Корреляционное обнаружение сигналов	11
3. Архитектура	14
3.1. Место ПЛИС-обнаружителя в системе	15
3.2. Формат данных и интерфейсы обнаружителя	16
3.3. Организация вычислительного тракта обнаружителя . .	18
4. Особенности реализации	20
4.1. Расчёт корреляции посредством БПФ	20
4.2. Определение частотной ошибки сигнала	31
5. Тестирование	39
5.1. Параметры тестовой симуляции	39
5.2. Организация моделирования	40
5.3. Уровни симуляции в Vivado	41
5.4. Выводы по результатам тестирования	42
Заключение	44
Список литературы	46

Введение

Современные системы связи и радиолокации распространены в разных областях деятельности человека. Речь идет о таких системах как сети высокоскоростного мобильного интернета 4G LTE и 5G или радиостанциях класса DMR, используемых для обеспечения сухопутной мобильной радиосвязи. Неотъемлемой частью таких систем является функция обнаружение сигналов, где под обнаружением понимается определение момента времени начала сигнала и его смещения по частоте относительно центральной частоты приема.

Данная задача является сложной в силу наличия в радиоэфире шумов и помех, неидеальностью приёмной и передающей аппаратуры, а также эффектов многолучевого распространения сигналов (доплеровский сдвиг частоты, искажение формы сигнала).

Оптимальным методом обнаружения сигналов на фоне аддитивного белого Гауссовского шума (АБГШ) является корреляционное обнаружение сигналов [28]. Основная идея этого метода заключается в том, что в сигнал на передающей стороне помещается специальная последовательность (преамбула), которая заранее известна как передатчику, так и приемнику. При приеме сигнала осуществляется вычисление корреляции между принятым сигналом и сохраненным в памяти эталонным образцом. В силу того, что корреляция двух сигналов является мерой их сходства, при появлении в принимаемом сигнале преамбулы значение корреляции резко возрастает, что позволяет идентифицировать начало полезного сигнала.

Для вычисления корреляционного интеграла при частоте дискретизации сигнала f_s и длине преамбулы в L отсчетов требуется выполнить $f_s * L$ операций умножения-сложения (multiply-accumulate, MAC) за секунду. Соответственно, для системы цифровой связи с частотой дискретизации 400 kHz и длиной преамбулы в 1024 отсчета для расчета корреляционного интеграла потребуется вычислитель с производительностью не менее 410 MIPS, что, в принципе, не является проблемой для современного аппаратного обеспечения.

Однако принимаемый сигнал содержит различные искажения, в числе которых особенно важна ошибка по частоте [15]. Наличие частотной ошибки в сигнале препятствует расчету корреляции, поскольку преамбула с частотной ошибкой не совпадает с эталонной и, следовательно, полученное значение корреляции будет ниже порогового. Таким образом, наличие частотной ошибки приводит к деградации корреляционного пика и делает обнаружение сигнала невозможным. В связи с этим эффект частотной ошибки должен быть нивелирован, что достигается путем определения величины частотной ошибки и выполнения частотной коррекции сигнала на найденную величину.

Необходимость компенсации частотной ошибки приводит к значительному увеличению вычислительной сложности метода корреляционного обнаружения сигналов. При переборе N различных частотных ошибок необходимо выполнить N раз расчет корреляционного интеграла, что приводит к увеличению объема вычислений в N раз (типичным значением N является 100). Таким образом, реализация метода корреляционного обнаружения сигналов для системы связи, рассмотренной ранее, потребует производительность вычислителя уже в 41000 MIPS, что является вызовом для современных вычислителей.

В качестве основного вычислителя для систем радиолокации и связи могут выступать цифровые сигнальные процессоры, интегральные схемы и ПЛИС [6]. Основными производителями сигнальных процессоров являются компании Texas Instruments и Analog Devices (США), которые выпускают семейства процессоров TMS320C64xx [22] и TigerSHARC [12] соответственно. Использование сигнальных процессоров оправдано при производстве мелкосерийных или средне серийных изделий, требующих для своей работы умеренную производительность. В случае крупносерийного производства экономически оправданным становится переход к использованию интегральных схем. В данном сценарии также появляется интерес к ПЛИС (например AMD 7 series FPGA [25] или Altera Cyclone IV FPGA [7]) как к средству прототипирования целевой интегральной схемы. Также выбор ПЛИС может быть обусловлен высокой вычислительной сложностью решаемой задачи, для которой произво-

дительности сигнального процессора будет недостаточно [9].

Одной из платформ, сочетающей интерфейс с радиоканалом (микросхема, обеспечивающая прием и передачу радиосигналов), ПЛИС и процессор, является технология программно-определяемой радиосистемы (Software Defined Radio, SDR) [21]. В рамках данной технологии реализация ряда компонентов системы связи выполняется средствами программными, а не аналоговыми, что позволяет осуществлять перенастройку широкого спектра параметров прямо во время работы SDR. Данным обстоятельством, в совокупности со способностью современных ПЛИС к выполнению частичной переконфигурации без нарушения работоспособности остальной части ПЛИС, объясняется высокая гибкость SDR-решений и интерес к использованию программно-определяемых радиосистем в качестве платформы для разработки и прототипирования систем цифровой связи.

В рамках технологии SDR реализация ресурсоемкого метода корреляционного обнаружения сигналов непосредственно на ПЛИС позволяет разгрузить центральный процессор системы для выполнения других задач, таких как демодуляция, декодирование, протокольная обработка и управление системой в целом.

АО “Концерн ГРАНИТ” занимается разработкой современных систем связи и радиоэлектроники для профильных министерств и ведомств Российской Федерации. В рамках работ по созданию беспроводных систем связи было необходимо разработать обнаружитель цифровых сигналов, работающий на ПЛИС. В качестве платформы для реализации обнаружителя было выдвинуто требование по использованию устройства Pluto SDR+, которое является клоном ADALM-PLUTO SDR [1] производства Analog Devices, и представляет собой доступную реализацию SDR на базе системы на кристалле (System on a Chip, SoC) Xilinx Zynq-7000 [27], сочетающую программируемую логику и двухъядерный процессор ARM.

1. Постановка задачи

Целью работы является разработка подсистемы для устройства Pluto SDR+, осуществляющей эффективную реализацию на уровне регистровых передач алгоритма обнаружения сигналов с цифровой модуляцией на ПЛИС с использованием языка программирования VHDL и аппробацией на устройстве. Для достижения данной цели были поставлены следующие задачи.

1. Выполнить обзор существующих и доступных алгоритмов обнаружения цифровых сигналов на ПЛИС, а также проблематики их реализации на ПЛИС. Выбрать подходящий алгоритм.
2. Спроектировать архитектуру ПЛИС-обнаружителя с учетом дальнейшей интеграцию в устройство Pluto SDR+.
3. Реализовать обнаружитель на языке VHDL, решив ряд следующих задач:
 - вычислительно эффективный расчет корреляции, использующий минимальный объем ресурсов ПЛИС;
 - бесшовный расчёт корреляции для непрерывного потока входных данных, поступающих в режиме реального времени;
 - проверка заданного набора гипотез о величине частотной ошибки входного сигнала.
4. Выполнить тестирование полученного решения.

2. Обзор

2.1. Особенности процесса разработки под ПЛИС

Программируемые логические интегральные схемы (ПЛИС) – это класс цифровых интегральных схем, логика которых не определяется на этапе производства, а конфигурируется пользователем после изготовления. ПЛИС состоит из набора логических блоков и множества программируемых соединений. Логические блоки содержат примитивы, такие как D-триггеры, таблицы перекодировки (Lookup Table, LUT), DSP-блоки, мультиплексоры, порты ввода-вывода и блоки памяти.

Важно отметить, что процесс конфигурирования ПЛИС существенно отличается от обычного процесса программирования, поскольку ПЛИС, в отличие от процессора, не выполняет последовательность инструкций. В случае ПЛИС код, написанный на языке описания аппаратуры (Hardware Description Language, HDL), описывает структуру и поведение цифровой схемы. На основе данного описания инструменты автоматизированного проектирования формируют конфигурацию внутренних ресурсов микросхемы, обеспечивающую реализацию заданного алгоритма.

Для конфигурирования ПЛИС используются HDL-языки — VHDL [], Verilog [], SystemC [], — а также программные средства (специальные IDEs), предоставляемые производителями микросхемы. Основной задачей таких средств является преобразование HDL-кода в конфигурацию логических примитивов, имеющих в ПЛИС, их размещение и маршрутизация соединений. В связи с этим основными компонентами таких IDE являются: редактор HDL-кода, синтезатор, средство размещения и трассировки (place-and-route), а также генератор конфигурационного файла (битового потока).

Разработчик может оказывать влияние на синтез и размещение при помощи ограничений (constraints), задающих временные, топологические и электрические параметры проекта.

Результатом разработки на языках HDL является описание аппарат-

ной структуры ПЛИС, которое может быть оформлено в виде повторно используемых модулей, называемых IP-ядрами. IP-ядро может использоваться как самостоятельно, так и входить в состав более сложных систем. При необходимости RTL-описание — RTL? — проекта может быть использовано в дальнейшем для разработки интегральной схемы специального назначения (Application-Specific Integrated Circuit, ASIC), предназначенной для серийного производства.

В настоящее время ПЛИС часто комбинируются с процессорным ядром на одном кристалле, создавая таким образом систему на кристалле (System-on-Chip, SoC). Такая система состоит из двух основных подсистем: процессорной системы (Processing System, PS) и программируемой логики (Programmable Logic, PL). Взаимодействие между подсистемами осуществляется по стандартизированным интерфейсам (например, AXI). Наличие процессорной подсистемы позволяет выполнять высокоуровневое управление, конфигурацию и отладку аппаратных модулей. Производительности процессорной системы, как правило, достаточно для развертывания операционной системы Linux[14, 26]. При отсутствии необходимости использования полноценной операционной системы возможно создание приложений типа Bare-Metal. Интеграция процессорной системы и программируемой логики расширяет функциональные возможности устройства и упрощает процесс верификации и отладки разрабатываемых аппаратных модулей. В связи с этим одним из распространённых сценариев применения SoC является организация платформ для разработки, отладки и апробации IP-ядер.

2.2. Используемые технологии

2.2.1. Платформа

Задачей разрабатываемого обнаружителя является обнаружение радиосигналов в заданном диапазоне частот и оценка их параметров. Для функционирования обнаружителя требуется входной цифровой сигнал, сформированный на основе принимаемого радиочастотного сигнала. Следовательно, аппаратная платформа должна обеспечивать не только вы-

числительные ресурсы ПЛИС, но и средства радиочастотного приёма и аналого-цифрового преобразования.

Таким образом, для реализации системы обнаружения требуется устройство, включающее систему на кристалле (SoC) и радиочастотный приёмопередающий тракт. Подобные устройства относятся к классу программно определяемых радиосистем (Software Defined Radio, SDR) [1], в которых значительная часть обработки сигнала выполняется программно или аппаратно-программными средствами.

В качестве аппаратной платформы была выбрана модифицированная версия устройства ADALM-PLUTO (Pluto SDR+) [2], в которой посредством изменения конфигурации расширен рабочий диапазон частот с 325 МГц–3,8 ГГц до 70 МГц–6 ГГц. Данное устройство сочетает в себе вычислительные ресурсы и радиочастотный тракт и обладает оптимальным соотношением стоимости и функциональных возможностей для решения поставленной задачи.

Основными компонентами выбранной платформы являются:

- система на кристалле AMD Zynq-7000 SoC, включающая процессорную подсистему и программируемую логическую часть;
- микросхема приёмопередатчика AD9363 [19], представляющая собой интегрированный RF-трансивер со встроенными аналого-цифровыми и цифро-аналоговыми преобразователями и цифровым интерфейсом передачи I/Q-отсчётов.

2.2.2. Используемые инструменты

При разработке проектов для ПЛИС выбор инструментов, во многом, определяется производителем целевой микросхемы. Как отмечалось ранее, код, написанный на языках HDL, не исполняется непосредственно устройством, а представляет собой описание структуры и поведения разрабатываемой цифровой схемы. На основе данного описания формируется конфигурация аппаратных ресурсов ПЛИС.

Преобразование RTL-описания в технологически зависимое представление, состоящее из примитивов целевой архитектуры, выполня-

ется инструментом синтеза. Процесс синтеза требует информации о составе и характеристиках ресурсов выбранной ПЛИС. После этапа синтеза выполняются процедуры размещения (placement) и трассировки (routing), а также статический временной анализ (Static Timing Analysis), позволяющий проверить корректность работы схемы с учётом заданных временных ограничений. Все перечисленные этапы требуют детального знания архитектуры устройства, поэтому официальные средства разработки предоставляются производителем ПЛИС.

В рассматриваемой работе используется система на кристалле семейства Zynq-7000, разработанная компанией AMD (ранее Xilinx). В качестве основной среды аппаратного проектирования применяется пакет Vivado Design Suite[4], предназначенный для синтеза, реализации и генерации конфигурационного файла.

Поскольку используемая платформа представляет собой SoC с процессорной подсистемой на базе ARM, для разработки программного обеспечения на языках C и C++ и его последующей загрузки и отладки используется среда AMD Vitis Unified Software Platform[3].

В качестве языка описания аппаратуры (Hardware Description Language, HDL) могут использоваться VHDL[23], Verilog[24] и SystemVerilog[13], которые получили широкое распространение в промышленной практике. Среда Vivado поддерживает данные языки. Для реализации обнаружителя в рамках данной работы был выбран язык VHDL, поскольку в организации АО концерн «ГРАНИТ» данный язык является основным.

2.3. Существующие решения

Разработанные IP-ядра зачастую обладают высокой коммерческой ценностью, поскольку на их основе могут быть созданы серийно производимые интегральные схемы. В силу этого значительная часть промышленных IP-ядер распространяется на проприетарной основе и недоступна для свободного использования и модификации.

В то же время существуют проекты и сообщества, разрабатывающие IP-ядра с открытым исходным кодом. К числу таких ресурсов относят-

ся специализированные каталоги и репозитории, например OpenCores[16], FuseSoC[11], а также агрегированные коллекции VHDL/Verilog IP-ядер[8]. Подобные решения, как правило, ориентированы на реализацию базовых интерфейсов, периферийных контроллеров или типовых функциональных блоков и могут иметь ограничения по уровню оптимизации, поддержке и адаптации под конкретные аппаратные платформы.

Дополнительно необходимо учитывать, что IP-ядро может быть реализовано с использованием вендор-специфичных примитивов и библиотек, что приводит к его несовместимости с ПЛИС другого производителя. Хотя RTL-описание в общем случае является переносимым, практическая реализация часто зависит от доступных аппаратных ресурсов, таких как блоки памяти, DSP-модули или специализированные интерфейсы. Возможен также сценарий, при котором IP-ядро не может быть реализовано на младших моделях ПЛИС того же семейства из-за недостаточного объёма доступных ресурсов.

Каждый из перечисленных факторов может служить обоснованием необходимости разработки собственного IP-ядра. В данной работе рассматривается создание собственного IP-ядра обнаружителя для платформы AMD Zynq-7000 SoC[5], поскольку проведённый обзор доступных открытых IP-ядер показал отсутствие готовых реализаций обнаружителя.

2.4. Корреляционное обнаружение сигналов

При обнаружении сигналов широко применяется подход, основанный на использовании преамбулы — заранее известной последовательности отсчётов, обладающей хорошими автокорреляционными свойствами.

Приёмник выполняет сравнение принятого сигнала с опорной преамбулой посредством вычисления взаимнокорреляционной функции, которая количественно характеризует степень сходства двух сигналов [?].

Взаимнокорреляционная функция для дискретных сигналов может

быть записана в виде

$$R_{xy}(k) = \sum_{n=0}^{N-1} x(n) y^*(n-k), \quad (1)$$

где $x(n)$ — принятый сигнал, $y(n)$ — опорная преамбула, $(\cdot)^*$ обозначает комплексное сопряжение, N — длина преамбулы.

При наличии сигнала максимум взаимнокорреляционной функции существенно превышает уровень отклика, формируемого шумом, что позволяет судить о наличии сигнала и его временном положении. Амплитуда корреляционного пика пропорциональна энергии сигнала.

Введём две гипотезы:

$$H_0 : x(n) = w(n),$$

$$H_1 : x(n) = s(n) + w(n),$$

где $w(n)$ — аддитивный белый гауссовский шум, $s(n)$ — искомый сигнал.

Решение об обнаружении принимается по правилу

$$\max_k |R_{xy}(k)| \underset{H_0}{\overset{H_1}{\geq}} \gamma, \quad (2)$$

где γ — порог обнаружения, выбираемый исходя из допустимой вероятности ложного срабатывания.

В отсутствии сигнала отклик корреляционного обнаружителя представляет собой случайную величину с известным распределением, что позволяет задать порог γ в соответствии с требуемой вероятностью ложной тревоги.

Корреляционный обнаружитель является оптимальным для канала с аддитивным белым гауссовским шумом при известной форме сигнала [?]. По этой причине в работе рассматривается разработка корреляционного обнаружителя.

Принятый радиосигнал отличается от переданного вследствие воздействия шумов приёмного тракта, внешних помех, эффектов многолучевого распространения, а также нестабильности опорных генераторов

передающей и приёмной сторон.

Поскольку корреляционный обнаружитель основан на когерентном суммировании отсчётов, наличие частотного рассогласования Δf между сигналом и опорой приводит к фазовому вращению во времени и снижению амплитуды корреляционного пика (рис. 1).

Поиск частотного смещения осуществляется дискретным перебором в пределах заранее известного диапазона. Непрерывный перебор невозможен, поэтому диапазон частот дискретизируется с некоторым шагом Δf_{step} .

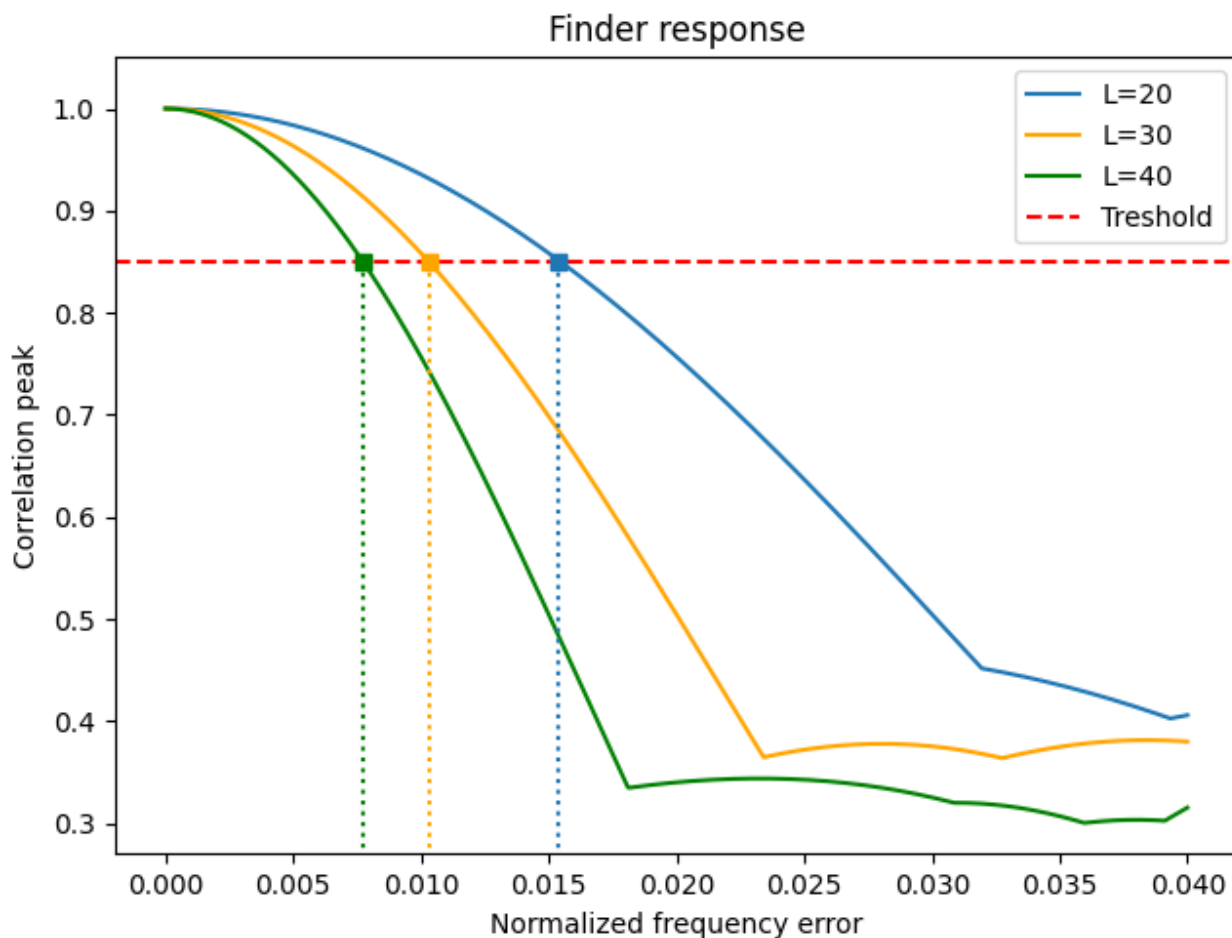
Величина шага связана с длительностью преамбулы. Чем больше длина преамбулы, тем уже главный лепесток функции неопределённости и тем меньше допустимая остаточная частотная ошибка.

Если шаг перебора слишком велик, остаточная частотная ошибка может привести к снижению корреляционного пика ниже порогового уровня и, как следствие, к пропуску сигнала. С другой стороны, уменьшение шага увеличивает число вычислительных операций и требования к вычислительным ресурсам.

Увеличение длины преамбулы приводит к росту её энергии при фиксированной мощности передатчика, что повышает дальность обнаружения сигнала. Однако увеличение длины преамбулы также повышает вычислительную нагрузку и усложняет процедуру частотного поиска.

Таким образом, выбор длины преамбулы представляет собой компромисс между энергетическими характеристиками обнаружителя и вычислительной сложностью его реализации.

Рис. 1: Влияние частотной ошибки на отклик корреляционного обнаружителя в зависимости от длины преамбулы.



3. Архитектура

Разработка ПЛИС-обнаружителя сигналов с цифровой модуляцией выполняется для платформы Pluto SDR+ []. Данная платформа представляет собой программно-определяемую радиосистему, в состав которой входят радиочастотный приёмопередатчик AD9363 [] и система на кристалле Zynq Z-7020 [] из семейства Zynq-7000 []. Приёмопередатчик AD9363 обеспечивает приём аналогового радиосигнала на несущей частоте, его перенос в основную полосу частот, оцифровку с заданной частотой дискретизации и формирование потока комплексных отсчётов сигнала. Система на кристалле Zynq Z-7020 объединяет процессорную систему на базе двухъядерного ARM Cortex-A9 [] и программируемую логику, реализованную на базе ПЛИС семейства Artix-7 [].

Как то бы надообосновать выбор этих технологий. Ну, или пояснить их свойства и как эти свойства используются в проекте. Чтобы это не было так сухо....

В качестве основы для аппаратной части системы используется открытый репозиторий HDL Reference Designs¹ компании Analog Devices. Репозиторий содержит HDL-библиотеки и проекты для различных эталонных и прототипных систем, а также исходный код на языках Verilog и VHDL и необходимые Tcl-скрипты для создания и сборки примеров проектов ПЛИС с использованием средств разработки Xilinx и Intel. Для платформы Pluto SDR+ используется проект Pluto², входящий в указанный репозиторий. Он включает описание аппаратной системы, файлы ограничений и Tcl-скрипты, необходимые для создания и сборки проекта ПЛИС.

В исходной архитектуре Pluto SDR+ процессорная система выполняет функции управления аппаратными компонентами и подготовки приемного тракта к работе. К таким функциям относятся настройка служебной периферии, необходимой для работы радиотракта и обмена данными, конфигурирование радиочастотного приёмопередатчика AD9363 по интерфейсу SPI, а также запуск DMA для передачи данных из ПЛИС в оперативную память процессорной системы.

По завершении настройки приёмного тракта AD9363 формирует поток комплексных отсчётов сигнала. Данный поток поступает в ПЛИС, проходит через внутренний цифровой тракт приёма и далее передаётся посредством DMA в оперативную память процессорной системы. В данной работе ПЛИС-обнаружитель встраивается в указанный тракт.

3.1. Место ПЛИС-обнаружителя в системе

Как отмечено выше, аппаратная часть работы основана на HDL-проекте Pluto SDR+ из репозитория Analog Devices³. Структура ПЛИС-прошивки данного проекта включает цифровой интерфейс радиоча-

¹<https://github.com/analogdevicesinc/hdl>

²<https://github.com/analogdevicesinc/hdl/tree/main/projects/pluto>

³<https://analogdevicesinc.github.io/hdl/projects/pluto/index.html>

стотного приёмопередатчика AD9363, приёмный и передающий тракты обработки отсчётов, DMA-блоки для обмена данными с оперативной памятью процессорной системы, а также служебные узлы управления, включая SPI и GPIO. В штатной конфигурации приёмного тракта поток комплексных отсчётов сигнала поступает из интерфейса AD9363 в ПЛИС, проходит цифровую обработку и передаётся через приёмный DMA-блок в DDR-память процессорной системы.

В предлагаемой архитектуре ПЛИС-обнаружитель размещается внутри приёмного тракта после блока RX_FIR_DECIMATOR и до входа приёмного DMA-блока. В этой точке сигнал уже прошёл цифровую фильтрацию и децимацию: поток отсчётов имеет заданную частоту дискретизации, а его спектр ограничен полосой, содержащей полезный сигнал.

Место встраивания обнаружителя в тракт приёма показано на рисунке 2. На вход обнаружителя поступает поток отсчётов сигнала после децимации, а в оперативную память через DMA передаются результаты корреляционной обработки. Процессорная система в данной схеме выполняет конфигурирование приёмного тракта и запуск DMA.



Рис. 2: Место ПЛИС-обнаружителя в приёмном тракте Pluto SDR+.

3.2. Формат данных и интерфейсы обнаружителя

Формат входных данных ПЛИС-обнаружителя соответствует формату потока, формируемого приёмным трактом AD9363. Разрядность АЦП AD9363 составляет 12 бит, поэтому действительная и мнимая компоненты комплексного отсчёта имеют разрядность по 12 бит. В тракте Pluto SDR+ каждая компонента передаётся с помощью 16-битной шины, при этом старшие 4 бита не содержат значащей информации. Следовательно, один комплексный отсчёт сигнала занимает 32 бита.

Выходные данные обнаружителя представляют собой поток комплексных значений взаимнокорреляционной функции. Поскольку обнаружение выполняется для четырёх частотных гипотез, каждому вре-

менному положению входного сигнала соответствует четыре комплексных значения корреляционного отклика. Одно комплексное значение корреляции представляется 32-битным словом, включающим 16-разрядную действительную и 16-разрядную мнимую части.

Обмен данными в приёмном тракте организован с использованием интерфейса AXI4-Stream, описанного в спецификации AMBA AXI-Stream Protocol Specification [2]. Данный интерфейс предназначен для однонаправленной потоковой передачи данных между аппаратными модулями и не использует адресную шину.

К основным сигналам интерфейса AXI4-Stream относятся:

- TDATA — шина данных;
- TVALID — признак наличия корректных данных на шине TDATA;
- TREADY — признак готовности приёмника принять данные;
- TLAST — признак завершения пакета данных.

Передача одного слова данных считается выполненной в такте, в котором одновременно активны сигналы TVALID и TREADY. Такой механизм синхронизации позволяет согласовать скорости работы соседних блоков: получатель может временно приостановить поток данных, а отправитель продолжает передачу после восстановления готовности последующего блока.

В рассматриваемой архитектуре интерфейс AXI4-Stream используется для передачи данных между приёмным трактом AD9363, ПЛИС-обнаружителем и блоком DMA, а также для обмена данными между компонентами самого обнаружителя. Через этот интерфейс передаются входные отсчёты сигнала, блоки данных для БПФ, спектральные значения, результаты спектрального умножения и выходные значения корреляционной функции. Границы блоков обработки задаются сигналом TLAST, что необходимо для согласования с IP-ядрами Быстрого Преобразования Фурье (БПФ), работающими с пакетами фиксированной длины.

Использование AXI4-Stream обеспечивает совместимость ПЛИС-обнаружи- с существующими IP-блоками проекта Pluto SDR+, включая тракт DMA. Благодаря этому обнаружитель встраивается в приёмный тракт без изменения общей схемы передачи данных из ПЛИС в оперативную память процессорной системы.

3.3. Организация вычислительного тракта обнару- жителя

Вычислительный тракт ПЛИС-обнаружителя организован как конвейер, работающий с последовательностью комплексных отсчётов сигнала. Можно выделить следующие основные стадии обработки.

Входные отсчёты → буферизация с перекрытием → БПФ →
спектральное умножение → обратное БПФ → удаление некорректных
отсчётов → выходные значения корреляции

Компонентная структура ПЛИС-обнаружителя показана на рис. 3. Диаграмма отражает основные аппаратные узлы тракта и связи между ними по потоковым интерфейсам: входную буферизацию с перекрытием, прямое БПФ, блок спектрального умножения с заранее подготовленными спектрами эталонной преамбулы, многоканальное обратное БПФ, узел удаления некорректных отсчётов и формирование выходного потока корреляционных значений.

Отсчёты сигнала поступают с приёмного тракта непрерывно, тогда как дальнейшая обработка выполняется блоками фиксированной длины. Поэтому на входе обнаружителя выполняется буферизация: входная последовательность накапливается до формирования блока, пригодного для подачи на БПФ. Для корректного вычисления линейной корреляции через спектральное умножение используется схема с перекрытием блоков или overlap-save. В каждом новом блоке сохраняется часть отсчётов предыдущего блока и добавляются новые отсчёты входного потока.

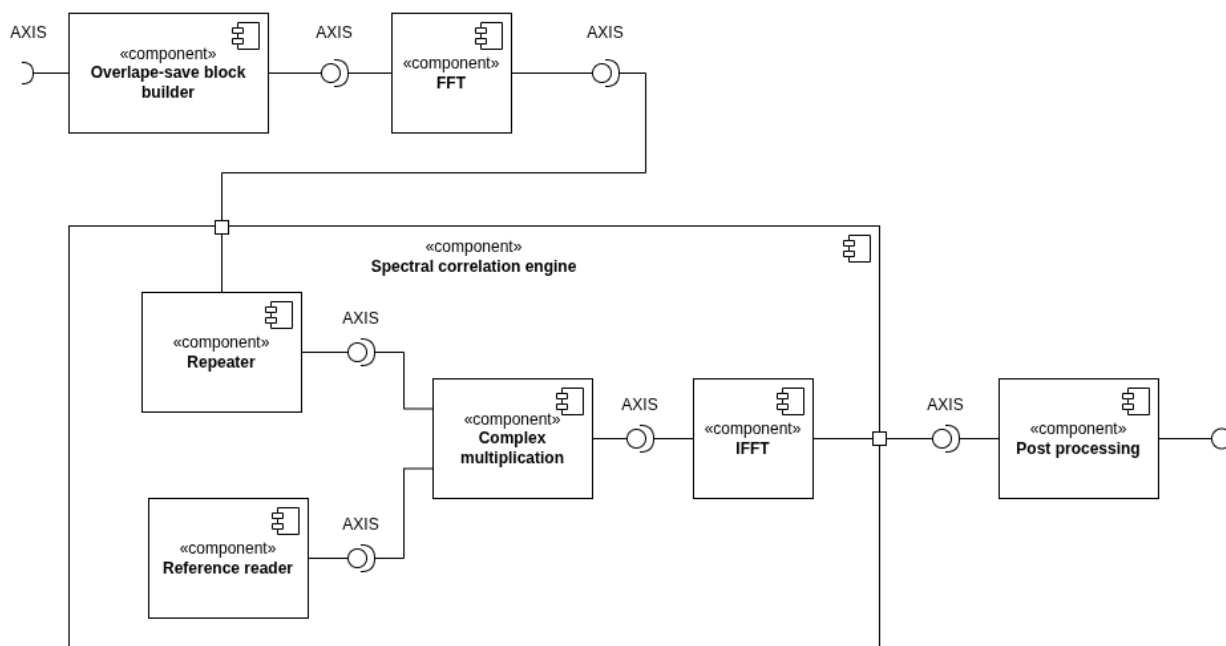


Рис. 3: UML-диаграмма компонентов ПЛИС-обнаружителя.

Сформированный блок поступает на IP-ядро БПФ, где переводится из временной области в частотную. Полученный спектр входного сигнала умножается на заранее подготовленные спектры эталонной преамбулы. Поскольку обнаружитель проверяет четыре частотные гипотезы, в тракте используются четыре варианта спектра преамбулы, соответствующие различным частотным смещениям.

После спектрального умножения для каждой частотной гипотезы выполняется обратное БПФ. На выходе формируются временные последовательности комплексных значений взаимнокорреляционной функции. Поскольку используется схема overlap-save, первые $K - 1$ отсчётов каждого выходного блока отбрасываются: они соответствуют некорректной области циклической свёртки. Оставшиеся отсчёты образуют корректный набор значений взаимнокорреляционной функции и передаются в выходной AXI4-Stream-поток.

4. Особенности реализации

4.1. Расчёт корреляции посредством БПФ

Основной вычислительно затратной операцией в ПЛИС-обнаружителе является вычисление взаимной корреляции входного сигнала с эталонной преамбулой. В данной работе обработка выполняется для комплексных отсчётов сигнала после децимации. Длина эталонной преамбулы составляет

$$K = 480.$$

Так как обнаружитель проверяет четыре частотные гипотезы, прямое вычисление корреляционной суммы требует большого числа комплексных умножений. Для снижения вычислительной сложности корреляция вычисляется через дискретное преобразование Фурье, реализуемое с помощью алгоритма БПФ.

4.1.1. Сведение корреляции к линейной свёртке

Пусть $x[n]$ обозначает последовательность комплексных отсчётов принятого сигнала, а $p[n]$ — эталонную преамбулу длиной K . Корреляционная сумма для положения преамбулы, начинающегося в отсчёте s , имеет вид

$$R_{xp}[s] = \sum_{m=0}^{K-1} x[s+m] p^*[m], \quad (3)$$

где $(\cdot)^*$ обозначает комплексное сопряжение.

Для сведения корреляции к свёртке введём ядро

$$h[n] = p^*[K-1-n], \quad n = 0, 1, \dots, K-1. \quad (4)$$

То есть ядро $h[n]$ получается из эталонной преамбулы разворотом во времени и комплексным сопряжением.

Линейная свёртка входной последовательности $x[n]$ с ядром $h[n]$

определяется выражением

$$y[n] = \sum_{q=0}^{K-1} h[q] x[n - q]. \quad (5)$$

Подставляя (4) в (5), получаем

$$y[n] = \sum_{q=0}^{K-1} p^*[K - 1 - q] x[n - q]. \quad (6)$$

После замены индекса $m = K - 1 - q$ выражение принимает вид

$$y[n] = \sum_{m=0}^{K-1} x[n - K + 1 + m] p^*[m]. \quad (7)$$

Сравнивая это выражение с (3), получаем

$$y[s + K - 1] = R_{xp}[s]. \quad (8)$$

Следовательно, вычисление корреляционной функции можно заменить вычислением линейной свёртки входной последовательности с ядром (4). Постоянный сдвиг индекса на $K - 1$ отсчётов не влияет на значение корреляционного пика, но должен учитываться при определении временной позиции обнаруженной преамбулы.

4.1.2. Вычисление свёртки через ДПФ и БПФ

Согласно теореме о свёртке, свёртке последовательностей во временной области соответствует умножение их спектров в частотной области [17, 18, 29]. Для конечных дискретных последовательностей это свойство используется в форме дискретного преобразования Фурье. При этом важно учитывать, что произведение N -точечных ДПФ соответствует циклической свёртке длиной N , а не линейной свёртке.

Пусть $x_N[n]$ и $h_N[n]$ — конечные последовательности длиной N . Их

N -точечные ДПФ определяются выражениями

$$X[k] = \sum_{n=0}^{N-1} x_N[n] e^{-j2\pi kn/N}, \quad k = 0, 1, \dots, N-1, \quad (9)$$

$$H[k] = \sum_{n=0}^{N-1} h_N[n] e^{-j2\pi kn/N}, \quad k = 0, 1, \dots, N-1. \quad (10)$$

После покомпонентного умножения спектров

$$Y[k] = X[k]H[k] \quad (11)$$

и выполнения обратного ДПФ

$$y_N[n] = \frac{1}{N} \sum_{k=0}^{N-1} Y[k] e^{j2\pi kn/N}, \quad n = 0, 1, \dots, N-1, \quad (12)$$

получается циклическая свёртка длиной N :

$$y_N[n] = \sum_{m=0}^{N-1} x_N[m] h_N[(n-m) \bmod N]. \quad (13)$$

Таким образом, ДПФ непосредственно реализует циклическую свёртку. Чтобы результат циклической свёртки совпал с линейной свёрткой двух конечных последовательностей, необходимо исключить циклическое наложение. Для этого последовательности дополняются нулями.

Если последовательности $x[n]$ и $h[n]$ имеют длины N_x и N_h , то их линейная свёртка имеет длину

$$N_y = N_x + N_h - 1. \quad (14)$$

Поэтому для вычисления линейной свёртки через N -точечное ДПФ необходимо выбрать длину преобразования

$$N \geq N_x + N_h - 1 \quad (15)$$

и дополнить обе последовательности нулями до длины N . В этом случае циклическая свёртка длиной N , полученная после обратного ДПФ от

произведения спектров, совпадает с линейной свёрткой.

В рассматриваемой задаче входной сигнал является непрерывным потоком отсчётов, поэтому однократное дополнение всей входной последовательности нулями неприменимо. Для потоковой обработки используются блочные методы, позволяющие получать линейную свёртку через последовательность циклических свёрток. В данной работе для этого применяется метод overlap-save, который рассматривается далее.

В аппаратной реализации ДПФ и обратное ДПФ не вычисляются напрямую по формулам (9)–(12). Для снижения вычислительной сложности используется алгоритм быстрого преобразования Фурье. Далее термин БПФ используется именно в этом смысле: как эффективный алгоритм вычисления ДПФ и обратного ДПФ.

В реализованной архитектуре используется БПФ длиной

$$N = 2048.$$

Ядро $h[n]$ имеет длину $K = 480$, поэтому перед вычислением спектра оно дополняется нулями до длины N :

$$h_N[n] = \begin{cases} h[n], & 0 \leq n < K, \\ 0, & K \leq n < N. \end{cases} \quad (16)$$

После этого заранее вычисляется спектр

$$H[k] = \sum_{n=0}^{N-1} h_N[n] e^{-j2\pi kn/N}, \quad k = 0, 1, \dots, N-1. \quad (17)$$

Поскольку преамбула известна заранее, разворот, комплексное сопряжение и вычисление спектра $H[k]$ выполняются вне ПЛИС. В аппаратной части спектральные коэффициенты хранятся в блочной памяти и считываются синхронно с выходным потоком прямого БПФ. Это позволяет исключить из аппаратного тракта операции разворота преамбулы и вычисления её ДПФ в реальном времени. Таким образом, аппаратная часть работает не с исходной преамбулой $p[n]$, а с заранее подготовленным набором спектральных коэффициентов $H[k]$.

В процессе работы обнаружителя для каждого входного блока вычисляется спектр $X[k]$, после чего выполняется покомпонентное умножение

$$Y[k] = X[k]H[k]. \quad (18)$$

Затем результат переводится обратно во временную область с помощью обратного БПФ:

$$y[n] = \text{IFFT}\{Y[k]\}. \quad (19)$$

В аппаратной реализации используется IP-ядро Xilinx Fast Fourier Transform, работающее с пакетами фиксированной длины и интерфейсом AXI4-Stream [10, 2]. Это накладывает два важных ограничения на организацию вычислительного тракта. Во-первых, входные отсчёты должны быть сгруппированы в блоки длиной N . Во-вторых, границы каждого блока должны быть явно заданы сигналом TLAST. Поэтому непрерывная последовательность отсчётов, поступающая после блока RX_FIR_DECIMATOR, перед подачей на БПФ преобразуется в последовательность перекрывающихся пакетов.

Кроме пакетного режима работы, при использовании IP-ядра БПФ необходимо учитывать масштабирование результата. В математической записи обратного ДПФ (12) используется нормировка множителем $1/N$. В IP-ядре Xilinx FFT масштабирование задаётся параметрами конфигурации и может распределяться по стадиям преобразования [10]. Поэтому абсолютное значение корреляционного отклика в аппаратной реализации зависит не только от энергии преамбулы, но и от выбранной схемы масштабирования. Это необходимо учитывать при выборе порога обнаружения и при сравнении результатов RTL-моделирования с программной моделью.

4.1.3. Метод overlap-save для непрерывного потока

Метод overlap-save применяется для вычисления линейной свёртки длинной или непрерывной последовательности с конечным ядром через ДПФ фиксированной длины [17, 20]. В данной работе он используется для того, чтобы непрерывный входной поток можно было обрабаты-

вать блоками длиной $N = 2048$, не теряя корректность корреляции на границах блоков.

Входной поток после блока `RX_FIR_DECIMATOR` поступает непрерывно. При этом БПФ выполняется над пакетами фиксированной длины N . Поэтому входная последовательность разбивается на перекрывающиеся блоки. Для r -го блока используется последовательность

$$x_r[n] = x[rL + n - (K - 1)], \quad n = 0, 1, \dots, N - 1, \quad (20)$$

где r — номер блока, а L — число новых входных отсчётов в одном блоке.

Каждый блок состоит из двух частей:

- первые $K - 1$ отсчётов — последние отсчёты предыдущего блока;
- следующие L отсчётов — новые отсчёты входного потока.

Так как полная длина блока равна N , получаем

$$N = (K - 1) + L. \quad (21)$$

Следовательно,

$$L = N - K + 1. \quad (22)$$

Для параметров данной работы

$$L = 2048 - 480 + 1 = 1569. \quad (23)$$

Таким образом, каждый входной пакет для БПФ состоит из 479 отсчётов перекрытия и 1569 новых отсчётов.

Для первого блока предыдущих отсчётов ещё нет, поэтому начальная область может быть заполнена нулями:

$$x[n] = 0, \quad n < 0. \quad (24)$$

Это соответствует начальному условию линейной свёртки.

Для каждого блока выполняется прямое БПФ:

$$X_r[k] = \text{FFT}\{x_r[n]\}. \quad (25)$$

Затем спектр входного блока умножается на заранее сохранённый спектр ядра:

$$Y_r[k] = X_r[k]H[k]. \quad (26)$$

После обратного БПФ получается результат циклической свёртки:

$$y_r[n] = \text{IFFT}\{Y_r[k]\}, \quad n = 0, 1, \dots, N - 1. \quad (27)$$

Покажем, почему в методе overlap-save первые $K - 1$ отсчётов результата являются некорректными. Так как ядро $h[n]$ имеет конечную длину K , циклическая свёртка может быть записана в виде

$$y_r[n] = \sum_{m=0}^{K-1} h[m] x_r[(n - m) \bmod N]. \quad (28)$$

Для первых отсчётов блока, когда $0 \leq n < K - 1$, часть индексов $n - m$ становится отрицательной. Из-за операции взятия по модулю N эти индексы заменяются индексами из конца текущего блока:

$$y_r[n] = \sum_{m=0}^n h[m] x_r[n - m] + \sum_{m=n+1}^{K-1} h[m] x_r[N + n - m], \quad 0 \leq n < K - 1. \quad (29)$$

Первое слагаемое соответствует обычной линейной свёртке, а второе является паразитной добавкой, возникающей из-за циклического переноса. Поэтому первые $K - 1$ отсчётов результата должны быть отброшены.

Для отсчётов с номерами

$$n = K - 1, K, \dots, N - 1$$

отрицательные индексы в сумме отсутствуют, поэтому циклическая свёртка совпадает с линейной:

$$y_r[n] = \sum_{m=0}^{K-1} h[m] x_r[n - m], \quad K - 1 \leq n < N. \quad (30)$$

Следовательно, на выход передаётся только фрагмент

$$\tilde{y}_r[\ell] = y_r[\ell + K - 1], \quad \ell = 0, 1, \dots, L - 1. \quad (31)$$

Если рассматривать $\tilde{y}_r[\ell]$ как отсчёт линейной свёртки, то его глобальный индекс равен

$$n_{\text{conv}} = rL + \ell. \quad (32)$$

С учётом соотношения (8) этому отсчёту соответствует корреляция для положения преамбулы

$$s = n_{\text{conv}} - (K - 1). \quad (33)$$

Из (32) видно, что корректные фрагменты соседних блоков стыкуются без пропусков и дублирования:

$$\begin{aligned} r\text{-й блок: } & rL, rL + 1, \dots, (r + 1)L - 1, \\ (r + 1)\text{-й блок: } & (r + 1)L, (r + 1)L + 1, \dots, (r + 2)L - 1. \end{aligned}$$

Поэтому метод overlap-save обеспечивает непрерывное вычисление свёртки, а следовательно и корреляционной функции, несмотря на то что БПФ обрабатывает данные блоками фиксированной длины.

С точки зрения аппаратной реализации это означает, что перед блоком БПФ должен быть сформирован пакет данных с заданной структурой: сначала $K - 1 = 479$ отсчётов перекрытия, затем $L = 1569$ новых отсчётов входного потока. После обратного БПФ, наоборот, необходимо удалить некорректную начальную часть результата и передать дальше только L корректных отсчётов. Поэтому метод overlap-save в вычислительном тракте реализуется не отдельной арифметической операцией, а схемой управления буферами и границами AXI4-Stream-пакетов.

В реализованном тракте это соответствует аппаратным операциям, представленным ниже.

1. Накопление новых отсчётов, поступающих после блока RX_FIR_DECIMATOR
2. Сохранение последних $K - 1 = 479$ отсчётов предыдущего блока.

3. Формирование пакета длиной $N = 2048$, состоящего из 479 сохранённых и 1569 новых отсчётов.
4. Подача сформированного пакета на вход прямого БПФ с формированием сигнала **TLAST**.
5. Умножение спектра входного блока на заранее рассчитанный спектр ядра корреляции.
6. Выполнение обратного БПФ.
7. Удаление первых $K - 1 = 479$ отсчётов результата.
8. Передача оставшихся $L = 1569$ Отсчётов в выходной AXI4-Stream-поток.

Необходимость буферизации следует из различия между непрерывным характером входного потока и пакетным режимом работы БПФ. Отсчёты от приёмного тракта поступают последовательно, тогда как IP-ядро БПФ начинает обработку только после формирования блока фиксированной длины. Поэтому в архитектуре используется схема двойной буферизации: пока один сформированный блок передаётся в тракт БПФ, другой буфер заполняется новыми входными отсчётами. При такой организации входной поток не прерывается, а перекрытие между соседними блоками формируется без потери данных на границах пакетов.

4.1.4. Оценка вычислительной сложности

Оценим выигрыш от использования БПФ по числу комплексных умножений. Комплексное умножение является наиболее затратной базовой операцией при аппаратной реализации, поскольку включает несколько действительных умножений и сложений. Поэтому сравнение по числу комплексных умножений достаточно наглядно показывает разницу между прямым вычислением корреляции и вычислением через БПФ.

Используем параметры реализованного обнаружителя:

$$K = 480, \quad N = 2048, \quad M = 4, \quad L = 1569,$$

где K — длина преамбулы, N — длина БПФ, M — число частотных гипотез, а L — число корректных отсчётов корреляционной функции, получаемых из одного блока методом overlap-save.

При прямом вычислении корреляции для одного положения преамбулы и одной частотной гипотезы требуется K комплексных умножений:

$$R_{xp}[s] = \sum_{m=0}^{K-1} x[s+m]p^*[m]. \quad (34)$$

Так как из одного блока необходимо получить L корректных значений корреляционной функции, а частотных гипотез используется M , общее число комплексных умножений при прямом расчёте равно

$$C_{\times, \text{direct}} = LKM. \quad (35)$$

После подстановки численных значений получаем

$$C_{\times, \text{direct}} = 1569 \cdot 480 \cdot 4 = 3\,012\,480. \quad (36)$$

Таким образом, прямой метод требует более трёх миллионов комплексных умножений на один блок новых входных данных.

Теперь оценим число комплексных умножений для метода, основанного на БПФ. Для одного входного блока выполняются:

- одно прямое БПФ длиной N ;
- умножение спектра входного блока на спектры ядра корреляции для M частотных гипотез;
- M обратных БПФ длиной N .

Для radix-2 БПФ длиной N число базовых операций типа “бабочка” равно $B_N = \frac{N}{2} \log_2 N$. В стандартной алгоритмической оценке каждая бабочка содержит одно комплексное умножение на поворачивающий

множитель. Поэтому число комплексных умножений для одного БПФ длиной N можно оценить как

$$C_{\times, \text{FFT}} = \frac{N}{2} \log_2 N. \quad (37)$$

Для $N = 2048$ имеем

$$\log_2 2048 = 11.$$

Следовательно,

$$C_{\times, \text{FFT}} = \frac{2048}{2} \cdot 11 = 11264. \quad (38)$$

В предложенной схеме выполняется одно прямое БПФ и четыре обратных БПФ. Поэтому суммарное число комплексных умножений, связанных с преобразованиями, равно

$$C_{\times, \text{FFT/IFFT}} = (M + 1) \frac{N}{2} \log_2 N. \quad (39)$$

Для $M = 4$ получаем

$$C_{\times, \text{FFT/IFFT}} = 5 \cdot 11264 = 56320. \quad (40)$$

Кроме самих БПФ, необходимо выполнить покомпонентное умножение спектра входного блока на спектры ядра корреляции для всех частотных гипотез:

$$Y_i[k] = X[k]H_i[k], \quad i = 0, 1, \dots, M - 1. \quad (41)$$

Для каждого спектрального отсчёта выполняется M комплексных умножений. Поэтому число комплексных умножений на этапе спектрального умножения равно

$$C_{\times, \text{spec}} = NM. \quad (42)$$

Для выбранных параметров:

$$C_{\times, \text{spec}} = 2048 \cdot 4 = 8192. \quad (43)$$

Итого число комплексных умножений для метода на основе БПФ

составляет

$$C_{\times, \text{FFT method}} = (M + 1) \frac{N}{2} \log_2 N + NM. \quad (44)$$

После подстановки численных значений:

$$C_{\times, \text{FFT method}} = 56320 + 8192 = 64512. \quad (45)$$

Таким образом, для одного блока новых входных данных прямой метод требует

$$3\,012\,480$$

комплексных умножений, а метод на основе БПФ —

$$64\,512$$

комплексных умножений.

Отношение этих величин равно

$$\frac{C_{\times, \text{direct}}}{C_{\times, \text{FFT method}}} = \frac{3\,012\,480}{64\,512} \approx 46,7. \quad (46)$$

Следовательно, по числу комплексных умножений метод на основе БПФ примерно в 47 раз эффективнее прямого вычисления корреляции.

4.2. Определение частотной ошибки сигнала

Одной из задач ПЛИС-обнаружителя является проверка заданного набора гипотез о величине частотной ошибки входного сигнала. Наличие частотного рассогласования между передатчиком и приёмником приводит к фазовому вращению отсчётов принятой преамбулы. Если это рассогласование не учитывать, когерентное суммирование в корреляционной сумме нарушается, а амплитуда корреляционного пика уменьшается.

Пусть частотная ошибка входного сигнала равна Δf . Тогда фрагмент принятого сигнала, соответствующий преамбуле, можно представить в виде

$$x[s + m] = A p[m] e^{j2\pi \Delta f (s+m) T_s} + w[s + m], \quad m = 0, 1, \dots, K - 1, \quad (47)$$

где A — комплексный коэффициент канала, $T_s = 1/f_s$ — период дискретизации, $w[n]$ — шумовая составляющая.

Если выполнять корреляцию только с исходной преамбулой $p[m]$, то при ненулевой частотной ошибке под знаком суммы остаётся фазовое вращение вдоль преамбулы. В результате отдельные слагаемые корреляционной суммы складываются не полностью когерентно, что приводит к уменьшению максимального значения корреляционного отклика.

Для компенсации этого эффекта используется набор частотных гипотез

$$\mathcal{F} = \{f_0, f_1, \dots, f_{M-1}\}, \quad (48)$$

где M — число проверяемых гипотез. В реализованной архитектуре

$$M = 4.$$

Для каждой гипотезы f_i формируется частотно-смещённая версия эталонной преамбулы:

$$p_i[n] = p[n]e^{j2\pi f_i n T_s}, \quad i = 0, 1, \dots, M - 1. \quad (49)$$

Корреляция входного сигнала с такой преамбулой имеет вид

$$R_i[s] = \sum_{m=0}^{K-1} x[s+m]p_i^*[m]. \quad (50)$$

Подставляя (49) в (50), получаем

$$R_i[s] = \sum_{m=0}^{K-1} x[s+m]p^*[m]e^{-j2\pi f_i m T_s}. \quad (51)$$

Если гипотеза f_i близка к истинной частотной ошибке Δf , остаточное фазовое вращение вдоль преамбулы становится малым, и корреляционный пик сохраняет большую амплитуду. Если же гипотеза существенно отличается от Δf , слагаемые в корреляционной сумме складываются с различными фазами, что приводит к снижению корреляционного отклика.

Поэтому оценка частотной ошибки может быть получена по номе-

ру гипотезы, для которой наблюдается наибольший корреляционный отклик:

$$\hat{i} = \arg \max_i \left(\max_s |R_i[s]| \right), \quad \hat{f}_{\text{err}} = f_{\hat{i}}. \quad (52)$$

Таким образом, задача проверки частотной ошибки сводится к одновременному вычислению нескольких корреляционных функций, каждая из которых соответствует своей частотной гипотезе.

4.2.1. Формирование частотных гипотез и спектров преамбулы

Как было показано в подразделе 4.1, корреляция вычисляется как линейная свёртка входной последовательности с ядром, полученным из преамбулы разворотом во времени и комплексным сопряжением. Для частотной гипотезы f_i это ядро имеет вид

$$h_i[n] = p_i^*[K - 1 - n], \quad n = 0, 1, \dots, K - 1. \quad (53)$$

С учётом (49) получаем

$$h_i[n] = p^*[K - 1 - n] e^{-j2\pi f_i(K-1-n)T_s}. \quad (54)$$

Именно набор ядер $h_i[n]$ определяет набор проверяемых частотных гипотез. Для каждого $h_i[n]$ заранее вычисляется N -точечное ДПФ:

$$H_i[k] = \sum_{n=0}^{N-1} h_{i,N}[n] e^{-j2\pi kn/N}, \quad k = 0, 1, \dots, N - 1, \quad (55)$$

где $h_{i,N}[n]$ — ядро $h_i[n]$, дополненное нулями до длины БПФ N .

В реализованной архитектуре используется

$$N = 2048.$$

Так как преамбула и набор частотных гипотез известны заранее, спектры $H_i[k]$ не вычисляются в ПЛИС в реальном времени. Они рассчитываются программно на этапе подготовки проекта и затем записываются в файлы инициализации памяти. В аппаратной части эти коэффициенты хранятся в блочной памяти BRAM.

Для каждого значения спектрального индекса k в памяти должны быть доступны коэффициенты

$$H_0[k], H_1[k], H_2[k], H_3[k],$$

соответствующие четырём частотно-смещённым преамбулам. При обработке входного спектра эти коэффициенты считываются последовательно по номеру гипотезы. Такой способ хранения позволяет использовать один и тот же вычислительный тракт для всех гипотез, не дублируя блоки прямого БПФ.

Предварительный расчёт спектров имеет два преимущества. Во-первых, из аппаратного тракта исключается генерация комплексных экспонент и вычисление ДПФ преамбулы. Во-вторых, изменение набора проверяемых частотных ошибок сводится к изменению содержимого памяти коэффициентов, а не к перестройке структуры корреляционного алгоритма.

4.2.2. Обработка спектра входного сигнала

После формирования входного блока методом overlap-save выполняется прямое БПФ. На выходе прямого БПФ формируется спектр входного блока:

$$X_r[k] = \text{FFT}\{x_r[n]\}, \quad k = 0, 1, \dots, N - 1, \quad (56)$$

где r — номер обрабатываемого блока.

Один и тот же спектральный отсчёт $X_r[k]$ используется для проверки всех частотных гипотез. Для каждой гипотезы выполняется покомпонентное умножение спектра входного блока на заранее рассчитанный спектр соответствующего ядра: $Y_{r,i}[k] = X_r[k]H_i[k]$, где $i = 0, 1, \dots, M - 1$.

В рассматриваемой реализации $M = 4$, поэтому для каждого номера спектрального отсчёта k необходимо сформировать четыре произведе-

ния:

$$\begin{aligned}Y_{r,0}[k] &= X_r[k]H_0[k], \\Y_{r,1}[k] &= X_r[k]H_1[k], \\Y_{r,2}[k] &= X_r[k]H_2[k], \\Y_{r,3}[k] &= X_r[k]H_3[k].\end{aligned}\tag{57}$$

Аппаратная реализация этого этапа построена как последовательная обработка набора гипотез для каждого спектрального отсчёта входного сигнала. Поток $X_r[k]$, поступающий с выхода прямого БПФ, сначала записывается в очередь. Далее отсчёты из этой очереди считываются с меньшей скоростью: один отсчёт $X_r[k]$ считывается один раз за M тактов и удерживается во внутреннем регистре на время проверки всех частотных гипотез.

Для фиксированного значения k последовательно перебираются номера гипотез $i = 0, 1, \dots, M - 1$. На каждом такте из BRAM считывается один спектральный коэффициент $H_i[k]$, соответствующий текущей частотно-смещённой преамбуле. После этого выполняется комплексное умножение $Y_{r,i}[k] = X_r[k]H_i[k]$.

Для случая $M = 4$ порядок вычислений для одного спектрального отсчёта имеет вид

$$\begin{aligned}X_r[k]H_0[k], \\X_r[k]H_1[k], \\X_r[k]H_2[k], \\X_r[k]H_3[k].\end{aligned}$$

После формирования четырёх произведений выполняется переход к следующему спектральному отсчёту $X_r[k + 1]$.

Такой способ организации позволяет переиспользовать один отсчёт спектра входного блока для всех частотных гипотез. В отличие от полностью параллельной схемы, здесь не требуется размножать поток $X_r[k]$ на четыре независимых тракта. Это уменьшает число параллельных умножителей и упрощает маршрутизацию данных, но увеличивает длительность этапа спектрального умножения в M раз относительно полностью параллельной реализации.

На выходе узла спектрального умножения для каждого номера k последовательно формируются четыре комплексных значения $Y_{r,0}[k]$, $Y_{r,1}[k]$, $Y_{r,2}[k]$, $Y_{r,3}[k]$. Так как многоканальное обратное БПФ ожидает данные по всем каналам одновременно, эти четыре результата должны быть сгруппированы в одно широкое слово AXI4-Stream. Для этого используется IP-блок AXI4-Stream Data Width Converter.

Блок AXI4-Stream Data Width Converter принимает четыре последовательных результата спектрального умножения и объединяет их в одно 128-битное слово: $\mathbf{Y}_r[k] = [Y_{r,0}[k], Y_{r,1}[k], Y_{r,2}[k], Y_{r,3}[k]]$.

При представлении одного комплексного значения 32-битным словом ширина такого вектора составляет $4 * 32 = 128$ бит. Сформированное 128-битное слово записывается в очередь, из которой затем считывается многоканальным ядром обратного БПФ. Очередь между AXI4-Stream Data Width Converter и обратным БПФ используется для согласования темпов работы узла спектрального умножения и IP-ядра обратного преобразования.

Таким образом, прямое БПФ входного блока выполняется только один раз, а его результат используется для всех частотных гипотез. Проверка гипотез выполняется последовательным переиспользованием каждого спектрального отсчёта $X_r[k]$, после чего результаты группируются в многоканальный формат для обратного БПФ.

4.2.3. Многоканальное обратное БПФ и формирование выходного потока

После спектрального умножения для каждого номера k сформированы четыре значения

$$Y_{r,0}[k], Y_{r,1}[k], Y_{r,2}[k], Y_{r,3}[k],$$

соответствующие четырём частотным гипотезам. Эти значения объединяются IP-блоком AXI4-Stream Data Width Converter в одно 128-битное слово и записываются в очередь. Из этой очереди данные считываются многоканальным IP-ядром обратного БПФ.

Таким образом, на вход обратного БПФ для каждого спектрального индекса k поступает вектор

$$\mathbf{Y}_r[k] = [Y_{r,0}[k], Y_{r,1}[k], Y_{r,2}[k], Y_{r,3}[k]]. \quad (58)$$

Каждый элемент этого вектора относится к своей частотной гипотезе. Многоканальное обратное БПФ выполняет независимое обратное преобразование для каждого канала:

$$y_{r,i}[n] = \text{IFFT}\{Y_{r,i}[k]\}, \quad i = 0, 1, \dots, M-1. \quad (59)$$

В результате на выходе обратного БПФ формируются четыре временные последовательности $y_{r,0}[n]$, $y_{r,1}[n]$, $y_{r,2}[n]$, $y_{r,3}[n]$. Каждая из них является корреляционным откликом для соответствующей частотной гипотезы.

Поскольку входные блоки формируются методом overlap-save, не все отсчёты результата обратного БПФ являются корректными. Как было показано в подразделе 4.1.3, первые $K-1 = 479$ отсчётов каждого блока содержат вклад циклического наложения и удаляются. На выход передаются только $L = 1569$ корректных отсчётов для каждой частотной гипотезы:

$$\tilde{y}_{r,i}[\ell] = y_{r,i}[\ell + K - 1], \quad \ell = 0, 1, \dots, L-1, \quad i = 0, 1, \dots, M-1. \quad (60)$$

Для каждого временного положения ℓ выход обнаружителя представляет собой вектор корреляционных откликов:

$$\mathbf{r}_r[\ell] = [\tilde{y}_{r,0}[\ell], \tilde{y}_{r,1}[\ell], \tilde{y}_{r,2}[\ell], \tilde{y}_{r,3}[\ell]]. \quad (61)$$

Дальнейшее определение частотной ошибки может выполняться по максимальному модулю элементов этого вектора. Если для некоторого временного положения s выполняется превышение порога обнаружения, то номер частотной гипотезы может быть найден как

$$\hat{i}[s] = \arg \max_i |R_i[s]|. \quad (62)$$

Соответствующая оценка частотной ошибки равна

$$\hat{f}_{\text{err}}[s] = \hat{f}_{i[s]}. \quad (63)$$

В рамках разработанной ПЛИС-подсистемы основная задача аппаратного тракта состоит в формировании всех корреляционных откликов для заданного набора гипотез. Последующее сравнение амплитуд, выбор максимума и принятие решения о наличии сигнала могут выполняться либо в аппаратной логике, либо в процессорной системе после передачи результатов через DMA. Такое разделение удобно для отладки: ПЛИС выполняет наиболее ресурсоёмкую часть обработки, а процессорная подсистема получает полный набор корреляционных функций и может изменять критерии обнаружения без перестройки аппаратного проекта.

5. Тестирование

Тестирование выполнялось для разработанного ПЛИС-обнаружителя как самостоятельного модуля. Объёмлющий проект приёмника для Pluto SDR+, включая тракт обмена с АЦП, DMA и процессорную систему, в данном разделе не рассматривался. Целью проверки было подтвердить корректность вычислительного тракта обнаружителя: формирование блоков overlap-save, прямое БПФ, обработку частотных гипотез, обратное БПФ и получение корреляционных откликов.

Проверка проводилась в среде Vivado. Результаты аппаратного моделирования сравнивались с программной моделью, реализованной на Python.

5.1. Параметры тестовой симуляции

Для моделирования был сформирован комплексный тестовый сигнал с частотой дискретизации $f_s = 100$ кГц, что соответствует режиму работы обнаружителя после децимации. В тестовый сигнал были добавлены три копии эталонной преамбулы, расположенные в разные моменты времени.

В сигнал была внесена частотная ошибка $f_{\text{err}} = -2800$ Гц.

Размер БПФ в архитектуре обнаружителя равен $N_{\text{FFT}} = 2048$. Соответствующее частотное разрешение составляет

$$\Delta f = \frac{f_s}{N_{\text{FFT}}} = \frac{100000}{2048} \approx 48,828125 \text{ Гц.} \quad (64)$$

Для проверки механизма частотных гипотез были выбраны четыре частотных сдвига:

$$[-2783,203125, -2734,375, -2685,546875, -2636,71875] \text{ Гц.}$$

Первый сдвиг является ближайшим к внесённой частотной ошибке. Остальные гипотезы последовательно удаляются от истинного значения, что позволяет проверить уменьшение корреляционного пика при росте остаточного частотного рассогласования.

Основные параметры теста приведены в таблице 1.

Параметр	Значение
Частота дискретизации f_s	100 кГц
Размер БПФ N_{FFT}	2048
Частотное разрешение Δf	48,828125 Гц
Внесённая частотная ошибка f_{err}	-2800 Гц
Количество преамбул в тестовом сигнале	3
Количество частотных гипотез	4

Таблица 1: Параметры тестовой симуляции обнаружителя

5.2. Организация моделирования

Спектры частотно-смещённых преамбул были предварительно рассчитаны в Python. На их основе сформированы файлы инициализации памяти формата `.coe`, используемые для загрузки коэффициентов в BRAM модуля `reference_reader`.

Входной тестовый сигнал также подготавливался программно и подавался на вход обнаружителя в тестовом окружении. В ходе моделирования проверялись следующие этапы:

- формирование перекрывающихся блоков длиной 2048 отсчётов;
- выполнение прямого БПФ;
- умножение спектра входного блока на спектры четырёх частотных гипотез;
- упаковка результатов спектрального умножения в многоканальный поток;
- выполнение обратного БПФ;
- удаление некорректной части результата overlap-save;
- формирование выходных корреляционных откликов.

Полученные в симуляции отсчёты корреляционных функций сохранялись во внешний файл и затем сравнивались с результатами программной модели.

5.3. Уровни симуляции в Vivado

В Vivado были выполнены уровни моделирования, представленные ниже.

1. Behavioral Simulation — функциональная RTL-симуляция до синтеза.
2. Post-Synthesis Functional Simulation — функциональная симуляция после синтеза.
3. Post-Synthesis Timing Simulation — временная симуляция после синтеза.
4. Post-Implementation Functional Simulation — функциональная симуляция после размещения и трассировки.
5. Post-Implementation Timing Simulation — временная симуляция после размещения и трассировки.

Первые четыре уровня симуляции показали корректную работу обнаружителя. На выходе формировались корреляционные отклики для всех четырёх частотных гипотез. Для гипотезы, ближайшей к внесённой ошибке — 2800 Гц, наблюдались три выраженных корреляционных пика, соответствующие трём преамбулам в тестовом сигнале. Для более удалённых гипотез амплитуда пиков уменьшалась, что соответствует ожидаемому влиянию остаточного частотного рассогласования.

Расположение пиков и относительное изменение амплитуд совпали с программной моделью. Это подтверждает корректность алгоритмической части обнаружителя и RTL-описания вычислительного тракта.

На уровне Post-Implementation Timing Simulation корректная работа получена не была. В этой симуляции значительная часть сигналов,

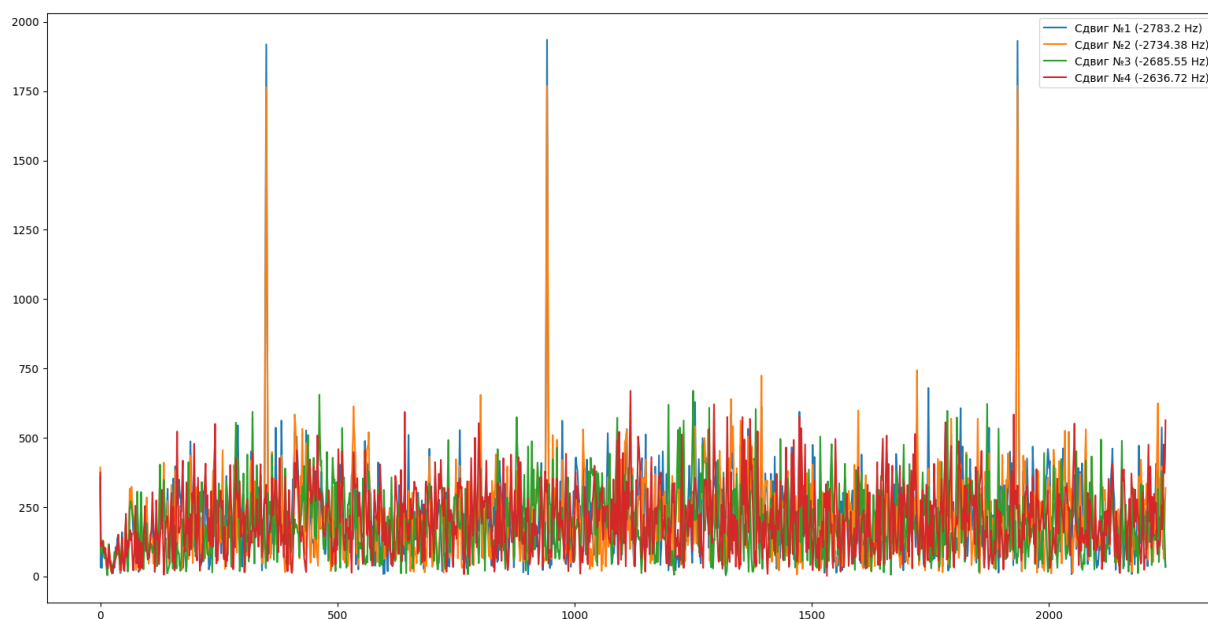


Рис. 4: Корреляционные функции для четырёх частотных гипотез

относящихся к IP-ядрам XFFT, принимала неопределённое значение X . Эти значения распространялись дальше по вычислительному тракту, из-за чего спектральное умножение и обратное БПФ не формировали корректные корреляционные отклики.

Такой результат означает, что функциональная корректность обнаружителя подтверждена, но временная модель размещённой реализации требует дополнительной отладки. Возможными направлениями дальнейшей проверки являются анализ сигналов сброса и инициализации XFFT, проверка временных ограничений, а также уточнение условий запуска post-implementation timing simulation для используемых IP-ядер.

5.4. Выводы по результатам тестирования

Проведённое тестирование подтвердило корректную работу разработанного обнаружителя на уровнях Behavioral Simulation, Post-Synthesis Functional Simulation, Post-Synthesis Timing Simulation и Post-Implementation Functional Simulation. Полученные корреляционные отклики совпадают с программной моделью по положению пиков и по зависимости амплитуды от выбранной частотной гипотезы.

Тестирование не охватывало полный проект приёмника для Pluto SDR+. Кроме того, работоспособность на уровне Post-Implementation Timing Simulation не была подтверждена из-за появления неопределённых значений X в сигналах IP-ядер XFFT. Поэтому в рамках данной работы верифицирована функциональная корректность обнаружителя, тогда как временная корректность размещённой реализации требует отдельного исследования.

Заключение

В ходе выполнения выпускной квалификационной работы получены следующие результаты.

1. Выполнен обзор существующих методов обнаружения цифровых сигналов и особенностей их реализации на ПЛИС. Установлено, что корреляционный метод является теоретически обоснованным и практически применимым для обнаружения сигналов с известной структурой преамбулы. Для аппаратной реализации выбран подход, основанный на вычислении взаимнокорреляционной функции с использованием быстрого преобразования Фурье в соответствии с теоремой о свёртке, что обеспечивает эффективность реализации.
2. Спроектирована архитектура ПЛИС-подсистемы обнаружения, предназначенной для интеграции в устройство Pluto SDR+. Разработанная архитектура реализует вычисление взаимнокорреляционной функции входного сигнала с эталонной преамбулой для набора частотных гипотез и учитывает особенности существующего цифрового тракта приёма и передачи данных между логической частью (PL) и процессорной системой (PS).
3. Реализован обнаружитель на языке VHDL, выполняющий:
 - эффективное вычисление корреляции из циклической свертки, рассчитываемой в спектральной области при помощи быстрого преобразования Фурье (БПФ);
 - бесшовное вычисление корреляции для потоковых данных за счёт реализации перекрытия пакетов отсчётов входного сигнала, поступающих на вход БПФ, и отбрасывания первых $K - 1$ отсчётов результата обратного БПФ;
 - одновременную проверку нескольких частотных гипотез путём умножения спектра входного сигнала на набор предрас-

считанных спектров частотных сдвигов преамбулы и использования многоканального обратного БПФ.

4. Выполнено тестирование разработанного ПЛИС-обнаружителя в среде Vivado без интеграции в полный проект Pluto SDR+. В первых четырёх уровнях симуляции подтверждена корректность расчёта корреляционных откликов для заданного набора частотных гипотез и совпадение результатов с программной моделью. В режиме Post-Implementation Timing Simulation корректная работа не была получена из-за появления неопределённых значений X в сигналах IP-ядер XFFT, поэтому временная корректность размещённой реализации требует отдельной отладки.

Список литературы

- [1] ADALM-PLUTO Evaluation Board [Electronic resource]. — URL: <https://www.analog.com/en/resources/evaluation-hardware-and-software/evaluation-boards-kits/adalm-pluto.html> (дата обращения: 1 ноября 2025 г.).
- [2] Arm. — AMBA AXI-Stream Protocol Specification. — URL: <https://developer.arm.com/documentation/ih0051/b/>.
- [3] AMD Vitis IDE. — URL: <https://www.amd.com/en/products/software/adaptive-socs-and-fpgas/vitis.html> (дата обращения: 6 января 2025 г.).
- [4] AMD Vivado IDE. — URL: <https://www.amd.com/en/products/software/adaptive-socs-and-fpgas/vivado.html> (дата обращения: 6 января 2025 г.).
- [5] AMD Zynq 7000. — URL: <https://www.amd.com/en/products/adaptive-socs-and-fpgas/soc/zynq-7000.html> (дата обращения: 6 января 2025 г.).
- [6] Adams, L. Choosing the Right Architecture for Real-Time Signal Processing Designs. White paper. Texas Instruments. 2002 [Electronic resource]. — URL: <https://www.ti.com/cn/lit/wp/spra879/spra879.pdf> (дата обращения: 1 ноября 2025 г.).
- [7] Altera Cyclone IV FPGA Device Family Overview [Electronic resource]. — URL: <https://cdrdv2-public.intel.com/654630/cyiv-51001.pdf> (дата обращения: 1 ноября 2025 г.).
- [8] Collection of open-source VHDL/Verilog IP cores. — <https://github.com/fabriziotappero/ip-cores>. — 2026. — Дата обращения: 2026.
- [9] FPGAs for High-Performance DSP Applications [Electronic resource]. — URL: https://cdrdv2-public.intel.com/650321/wp_dsp_comp.pdf (дата обращения: 1 ноября 2025 г.).

- [10] Fast Fourier Transform v9.1 LogiCORE IP Product Guide. — URL: https://www.xilinx.com/support/documents/ip_documentation/xfmt/v9_1/pg109-xfmt.pdf (дата обращения: 18 июня 2025 г.).
- [11] FuseSoC Core Directory. — <https://cores.fusesoc.net/>. — 2026. — Дата обращения: 2026.
- [12] General-Purpose TigerSHARC Processor Product Brief [Electronic resource]. — URL: [http://analog.com/media/en/news-marketing-collateral/product-highlight/4667563674187gptigersharc\(b\)final.pdf](http://analog.com/media/en/news-marketing-collateral/product-highlight/4667563674187gptigersharc(b)final.pdf) (дата обращения: 1 ноября 2025 г.).
- [13] IEEE Standard for SystemVerilog [Electronic resource]. — URL: <https://ieeexplore.ieee.org/document/10458102> (дата обращения: 1 ноября 2025 г.).
- [14] Linaro Linux. — URL: <https://old.linaro.org/core-technologies/linux-kernel/> (дата обращения: 6 января 2025 г.).
- [15] Mengali Umberto. Synchronization Techniques for Digital Receivers. — Springer, 1997.
- [16] OpenCores: Open Source IP Cores. — <https://opencores.org/>. — 2026. — Дата обращения: 2026.
- [17] Oppenheim Alan V., Schaffer Ronald W. Discrete-Time Signal Processing. — 3 edition. — Upper Saddle River, NJ : Prentice Hall, 2010.
- [18] Proakis John G., Manolakis Dimitris G. Digital Signal Processing: Principles, Algorithms, and Applications. — 4 edition. — Upper Saddle River, NJ : Pearson Prentice Hall, 2006.
- [19] RF transceiver AD9363. — URL: <https://www.analog.com/en/products/ad9363.html> (дата обращения: 6 января 2025 г.).

- [20] Smith Steven W. The Scientist and Engineer's Guide to Digital Signal Processing. — San Diego, CA : California Technical Publishing, 1997.
- [21] Software-Defined Radio Solutions from Analog Devices [Electronic resource]. — URL: <https://www.analog.com/media/en/news-marketing-collateral/solutions-bulletins-brochures/software-defined-radio-solutions-from-adi.pdf> (дата обращения: 1 ноября 2025 г.).
- [22] TMS320C64x Technical Overview [Electronic resource]. — URL: <https://www.ti.com/lit/pdf/spru395> (дата обращения: 1 ноября 2025 г.).
- [23] VHDL 2019 standard. — URL: <https://ieeexplore.ieee.org/document/8938196> (дата обращения: 6 января 2025 г.).
- [24] Verilog 2005 standard. — URL: <https://ieeexplore.ieee.org/document/1620780> (дата обращения: 6 января 2025 г.).
- [25] Xilinx 7 Series FPGAs Data Sheet: Overview [Electronic resource]. — URL: <https://docs.amd.com/v/u/en-US/ds190-Zynq-7000-Overview> (дата обращения: 1 ноября 2025 г.).
- [26] Xilinx Linux Kernel. — URL: <https://github.com/Xilinx/linux-xlnx> (дата обращения: 6 января 2025 г.).
- [27] Zynq-7000 SoC Data Sheet: Overview [Electronic resource]. — URL: <https://docs.amd.com/v/u/en-US/ds190-Zynq-7000-Overview> (дата обращения: 1 ноября 2025 г.).
- [28] Гришин Ю. П., Ипатов В. П., Казаринов Ю. М. Радиотехнические системы / Ed. by Ю. М. Казаринов. — Москва : Высшая школа, 1990.
- [29] Лукин А. Н. Введение в цифровую обработку сигналов (математические основы). — Москва : Лаборатория компьютерной графики и мультимедиа, МГУ, 2007.